

How the load-balancer Hive cell turns Kubernetes Services & EndpointSlices into eBPF LB-map entries, the call flow that picks a service and its backends, the cell hierarchy, and the changes made so it works against eBPF-for-Windows (efw).

1 Cell hierarchy

How loadbalancer.Cell composes writer / reflectors / maps / reconciler.

2 Code flow

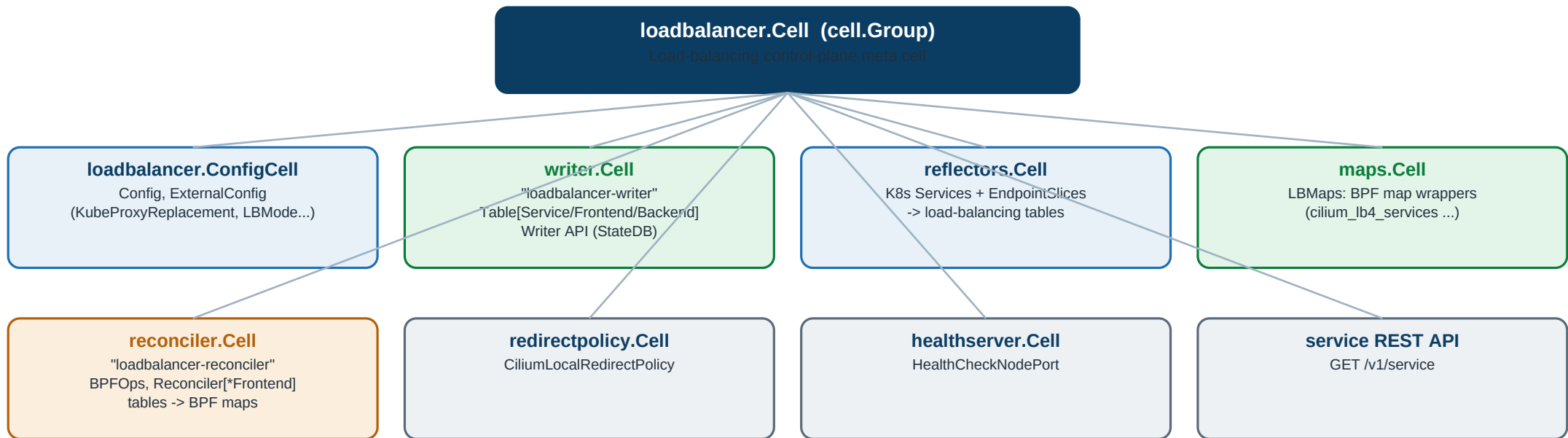
K8s objects -> StateDB tables -> BPF maps; where service & backend info is chosen.

3 Call flow

BPFOps.Update -> updateFrontend: service ID, backends, slots, master, rev-nat.

4 Windows BPF changes

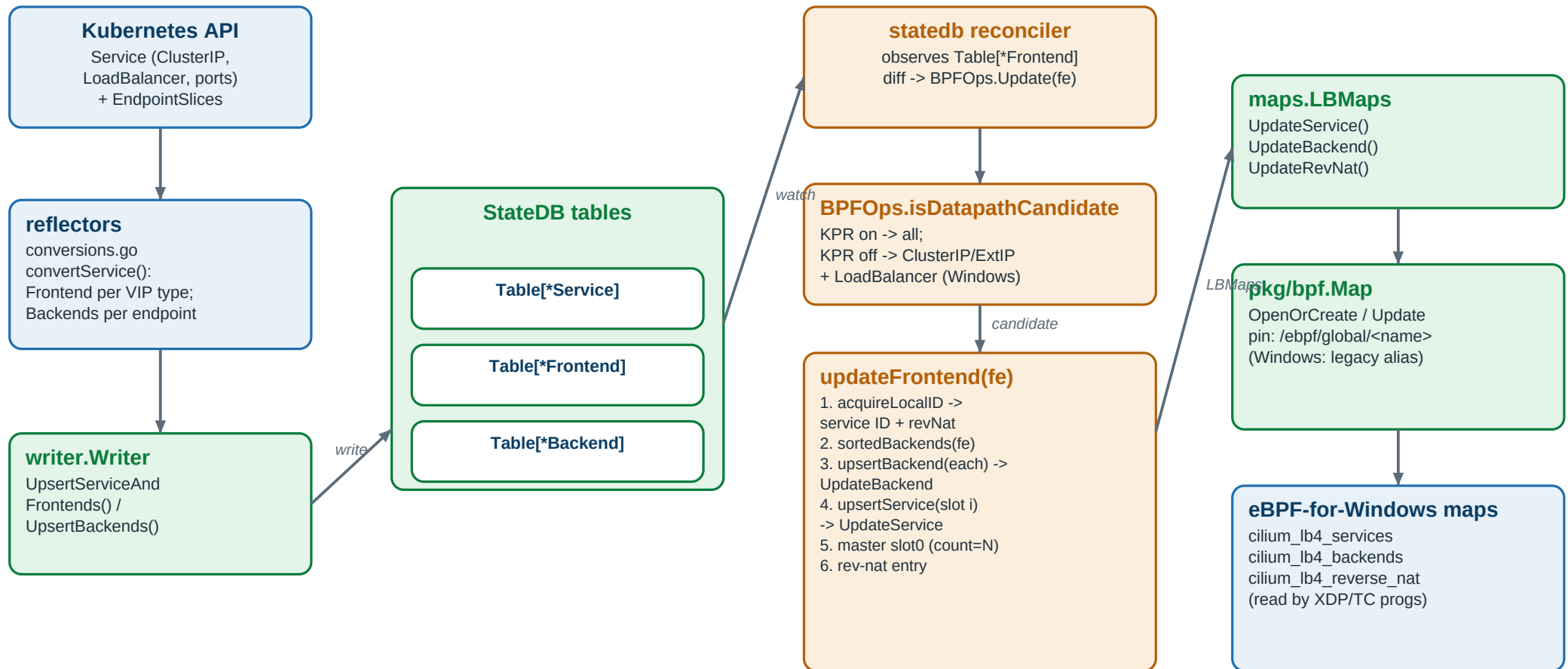
Pin path, legacy map-name alias, and LoadBalancer datapath candidate.



Green = data model & maps Amber = reconciliation Blue = config/reflect Grey = auxiliary

2 Code flow — picking service & backends and writing the maps

reflectors -> writer -> StateDB -> reconciler -> LBMaps



1. Reconciler.Update(txn, fe)

statedb reconciler delivers a changed *Frontend

2. ops.isDatapathCandidate(fe)

KPR off: ClusterIP/LocalRedirect/ExternalIPs + platformDatapathCandidate (LoadBalancer on Windows). Else deleteFrontend.

3. ops.updateFrontend(fe, isLocalAddr)

idempotent programming of one frontend into the maps

4. serviceIDAlloc.acquireLocalID(addr)

allocate/lookup Service ID -> also used as rev_nat index

5. NewService4Key / Service4Value; NewSvcFlag(...)

build key (VIP,port,proto,scope,slot) & value flags (svc type, ext/int policy, affinity, routable)

6. ops.sortedBackends(fe)

deterministic ordered backend list from Table[*Backend] (active first)

7. for each backend: ops.upsertBackend(id, be)

LBMaps.UpdateBackend -> cilium_lb4_backends[id] = {addr,port,proto}

8. for slot i=1..N: ops.upsertService(key.slot=i, backendID)

LBMaps.UpdateService -> cilium_lb4_services[VIP,port,slot i] = backendID

9. master: ops.upsertService(slot 0, count=N, revNat)

LBMaps.UpdateService -> cilium_lb4_services[VIP,port,slot 0] = {count,revNat,flags}

10. ops.upsertRevNat(revNat, addr)

LBMaps.UpdateRevNat -> cilium_lb4_reverse_nat[revNat] = {VIP,port}

11. LBMaps -> pkg/bpf.Map.Update -> ebpf-go -> efw

written at pin path /ebpf/global/<name>; XDP/TC datapath programs read these maps

4 Windows BPF changes

what made the agent share the datapath's efw maps

A. Map pin path

MapPath() -> agentMapPath(name)
Linux : /sys/fs/bpf/tc/globals/<n>
Windows: /ebpf/global/<n>
files: bpffs.go, bpffs_windows.go,
bpffs_linux.go

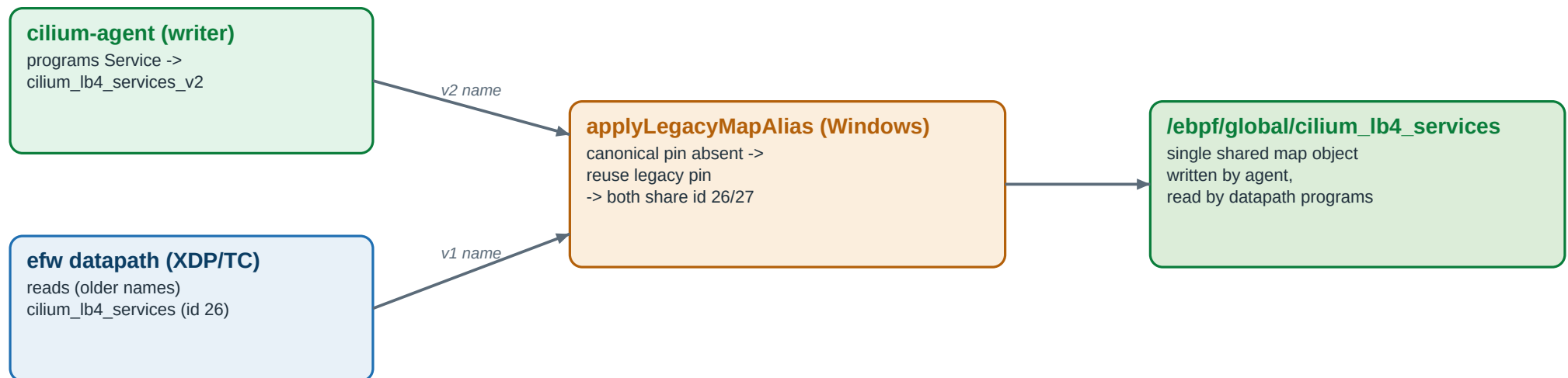
B. Legacy map-name alias

openOrCreate -> applyLegacyMapAlias
prefer datapath's existing v1 pin:
cilium_lb4_services_v2 ->
cilium_lb4_services (26/27)
files: map_legacy_windows.go,
map_legacy_other.go (no-op)

C. LoadBalancer datapath candidate

isDatapathCandidate ->
platformDatapathCandidate(fe)
KPR off + Windows: program
SVCTYPELoadBalancer VIPs
files: datapath_candidate_windows.go
datapath_candidate_other.go

Why it works: control plane and datapath must resolve to the SAME efw object



Guard: only maps with identical key/value geometry are aliased (services/backends). ipcache is NOT aliased (value size differs) to avoid a destructive unpin/recreate of the datapath map.